```
begin
    using HypertextLiteral
    using PlutoUI
    using PlutoTeachingTools
    using PlutoExtras
    using PlutoTest
    using ShortCodes
end
```

1 sec

```
using CairoMakie
```

8.1 sec

```
using PlutoLinks
```

1.5 sec

```
using PlutoSlides
```

46.2 ms

```
import MarkdownLiteral: @markdown
```

426 ms

```
beamer_color = "#ff7f50"
```

21.8 µs

```
PlutoSlides.slide_mode_settings(footer_left="David Métivier", footer_center="Julia for scientific computing and research",
color_subtitle_bg=beamer_color, h3_title=true, font_size=21,
    max_width="98%"
)
```

906 ms

Slide Mode

```
slide_mode_button()
```

24 µs

Présentation générale Julia et de quelques écosystèmes & Création de packages

David Métivier
INRAE - Montpellier





[credit]:

> This talk was partly inspired by Guillaume Dalle Julia talk + Valentin Churavy + Tim Holy lectures + videos and link shown in the notebook

How I Started Using Julia

Ph.D. *Laboratoire J.A. Dieudonné*
2014-2017 with Julien Barré

Kuramoto model
Coupled Oscillators
Synchronization
Nonlinear dynamics
Bifurcation

Electrical oscillators
Mean Field Control
Landau damping
Power systems
Smart Grids

Vlasov equation
Mean-Field
PDE
Bifurcation
Long range

Stats. for Climate/Environment
Stochastic Weather Generators
Climate risks in agronomy
Water quality forecasting
Physics informed Deep Learning
Robust Statistics
Software development

Uncertainty Quantification
Polynomial Chaos
Sparse
Power systems
Renewable energy

Robust Quasi Monte Carlo

INRAE MISTEA
2022–Now

Stochastic Weather Generator
Hidden Markov Models
Climate change / Risks

Post-Doc *Centre de Mathématiques Appliquées*
2020-2022 with Emmanuel Gobet

Post-Doc *Los Alamos National Laboratory*
2017-2020 with Michael Chertkov

Coding experience

- Matlab/Mathematica (Internship)
- C (with GPU) with Gnuplot/xmgrace (School and PhD)
- Very little Python (School and PhD)
- Julia (Los Alamos Postdoc 1 2018)

Package

- ExpectationMaximization.jl (*Developer*)
- QuasiMonteCarlo.jl (**Contributor**)
- StochasticWeatherGenerators.jl (*Developer*)
- PlutoSlides.jl (*Developer*)
- Smaller WIP projects RobustMeans.jl + ABCMethods.jl + PeriodicHiddenMarkovModels.jl (*Developer*)
- small contributions

Goal of the presentation

Program

1. **Motivation** for Julia
2. **Basics**
3. **Examples**: reference ecosystems and composability
4. **Writing a package** – writing & assembling four of them to train $\frac{\mathrm{d}u}{\mathrm{d}t} = \mathrm{NeuralNetwork}(u)$.

Goal: Give you a feeling that this is a **great modern tool for complex scientific problems** and teaching, and that **the language favors collaborative work**.

Later this week

- **Multiple dispatch**
- **Performance**
- **Parallelism**: threads, MPI, GPU – one lecture each
- **Interface** with other languages

Disclaimer

I will present many things: **a lot already exists in other languages**, sometimes as add-ons or wrapped C/C++ libraries.

Julia language and most of its packages are inspired by many cool existing things.

In this talk I want to emphasize the simplicity of some features and how far they can take us.

The Julia story

Thoughts by Gaël Varoquaux in May 2010 (co-founder of scikit-learn)



Gaël Varoquaux

Thu 20 May 2010

←Home

Personal views on scientific computing

under science academia scientific computing selected scientific software Read time: 3 min.

My contributions to the scientific computing software ecosystem are motivated by my vision on computational science.

Scientific research relies more and more on computing. However, most of the researchers are not software engineers, and as computing is becoming ubiquitous, the limiting factor becomes more and more the **human factor** [G. Wilson, 2006] [P. Norvig, 2009].

To address the needs of computing accross scientific fields, I believe that we need a **general-purpose**, **high-level**, **interactive**, and **highly-readable** language and set of tools for scientific computing.

- C does not answer my needs: does a molecular biologist know about pointers? Should she?

My take

I would add **speed/performance** to these requirements

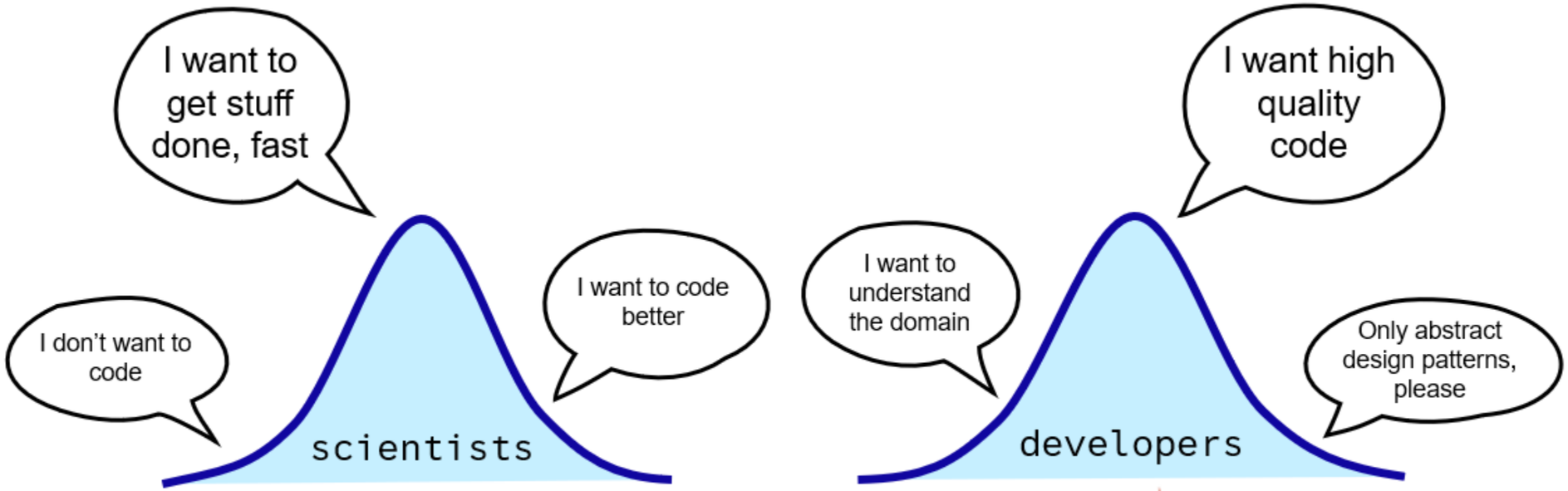
Why Julia was created?

> We are power **Matlab** users. Some of us are **Lisp** hackers. Some are **Pythonistas**, others **Rubyists**, still others **Perl** hackers. There are those of us who used **Mathematica** before we could grow facial hair. There are those who still can't grow facial hair. We've generated more **R** plots than any sane person should. **C** is our desert island programming language.
>
> **We love all of these languages**; they are wonderful and powerful. For the work we do — **scientific computing, machine learning, data mining, large-scale linear algebra, distributed and parallel computing** — each one is perfect for some aspects of the work and terrible for others. **Each one is a trade-off**.
>
> **We are greedy: we want more.**
>
> **Why we created Julia – Jeff Bezanson, Stefan Karpinski, Viral B. Shah, and Alan Edelman (2012)**

> Julia is designed to be easy and fast and questions notions generally held to be "laws of nature" by practitioners of numerical computing:
>
> 1. High-level dynamic programs have to be slow.
> 2. **One must prototype in one language and then rewrite in another language for speed or deployment.**
> 3. There are parts of a system appropriate for the programmer, and other parts that are best left untouched as they have been built by the experts.
>
> **Julia: A Fresh Approach to Numerical Computing – Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah (2017)**

The Two Languages Problem

Scientists vs Developers (1/2)



From My Target Audience by Matthijs Cox (2023)

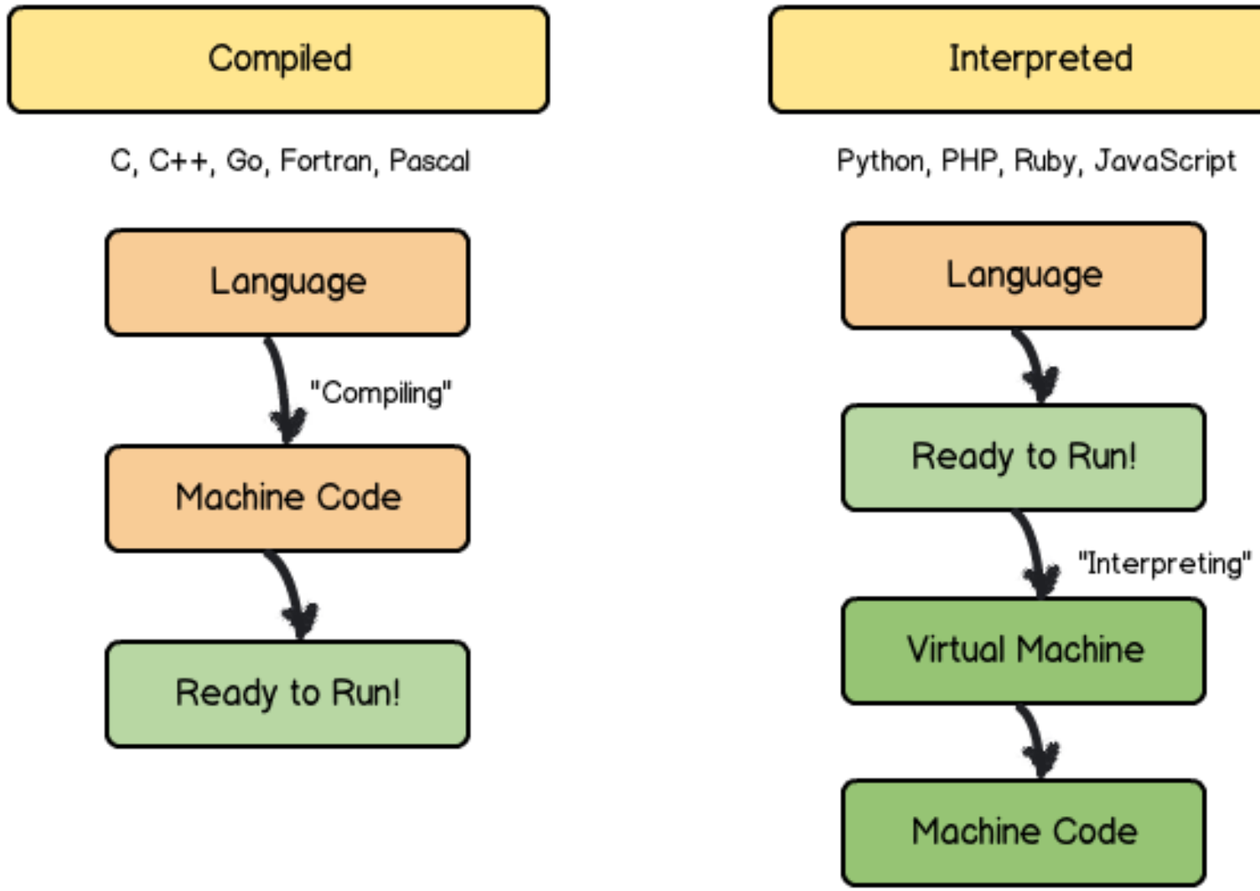
Scientists vs Developers (2/2)





From My Target Audience by Matthijs Cox (2023)

Compiled vs Interpreted



Compilation

- **Ahead of time** (C, C++, Go, Rust, ...): types are statically defined and the code is compiled before execution
- **Just-In-Time** (Julia, numba, JAX...): types inferred dynamically and functions compiled at first execution
- **Runtime** (Python, R, Matlab, ...): types inferred dynamically and functions compiled during runtime

Julia is **dynamically** typed, feels like a scripting language, and has good support for interactive use. Julia doesn't require you to annotate types: it infers them from the types of the arguments.

Just-In-Time

Just-In-Time (JIT)

Julia was designed **from the ground up** to allow inference to work well most of the time, not just within a single function but from the entry point of a computational pipeline to its terminus.

JIT in other languages

JIT is native to Julia.

Matlab and Python e.g. `numba` @jit, sometimes through **add-ons**, attempt to do the same thing, and when it works you get pretty good performance. But it places a lot of requirements on your code and works only in certain circumstances.

`JAX` now has "native" JIT.

Time To First X (TTFX)

The first time you do something in Julia, it takes a while.

The landmark problem: Writing a sum – Loops vs Vectorization

At school

```
s = 0
for i in 1:N
  s = s + x[i]
end
return s
```

The algorithm, written as the algorithm.

In Python/R/Matlab: don't.

> the loop is hidden from you in C. **Learning to use vectorized operations is a key skill in R**.
>
> **Monash R lesson**

> Experienced Matlab users like to say **Life is too short to spend writing for loops.**
>
> **1998 Getting Started with Matlab manual**

In Julia: do

```
s = 0
for i in 1:N
  s = s + x[i]
end
return s
```

Same code, *"as fast as C"*, and generic on top.

The middle column is the two-language problem

A vectorized `sum` **is** a loop – written in C, by somebody else.

- *for loops are slow* was never about loops: in a dynamic language the types are unknown, so **every iteration re-infers them**.
- The fast path therefore lives in **another language**, and you do not get to touch it.
- Julia compiles **your** loop for the types it actually sees ⇒ one language, nothing hidden underneath.

What if I cannot vectorize?

Sometimes you simply cannot

Complex dependencies on past elements – a Markov chain, a recursive filter, a neighbor average – and vectorizing is either impossible or an afternoon gone.

`[example]` Matlab example for averaging pixels; see here for context and here for the original post.

> "I remember working on an **R script that needed to loop through 33 million rows** of data, doing a complicated lag/join/basket kind of computation that would take 18 hours to run. Literally during one of these 18 hour runs, I saw the Julia announcement post and was immediately desperate for the kind of simple performance it promised. **I read the initial manual over a weekend, rewrote my script the following Monday morning, and it ran in 5 minutes.**"
>
> **Jacob Quinn (2022)**

Conclusion

- Loops are not slow – **re-inferring types at every iteration** is.
- Vectorizing is not always possible, and it is a time sink.
- Everywhere else the cure is to **rewrite in another language**: that is the two-language problem.
- In Julia the loop *is* the fast version.

Development burden

The usual recipe

Performance-critical packages are written in **C/C++/Fortran** for speed, then wrapped in **Python/R/Matlab** for usability.

PyTorch, TensorFlow, Pandas, NumPy, SciPy... all rely on a compiled core, and so do many smaller domain packages.

The cost of the two-language split

- Hard to **read** the source code
- Hard to **contribute** (who knows both C++ *and* the wrapper language?)
- Hard to **add features** or maintain
- Hard to **install** and **reproduce** results
- Poorly **composable** outside the package

A lot of effort

- Writing **glue code** between languages costs engineers, postdocs and years (**before LLMs**)
- Many packages **duplicate** the same features instead of sharing code
- Each extra language, compiler and dependency multiplies **compatibility** problems

Question:

Who never ran into **compatibility** or installation issues with Python/R?

The more dependencies (especially across languages and compilers), the more issues with reproducibility, compatibility and installation

Of course, this does not mean you won't have any with Julia.

Julia Basics

Website

julia Download Docs Learn Blog Community Contribute JSoC ☆ Star ♡ Sponsor

The Julia Programming Language

Install v1.13.1 Docs Community ☆ Star 49,207

Fast

Julia was designed for high performance. Julia programs automatically compile to efficient native code via LLVM, and support multiple platforms.

Dynamic

Julia is dynamically typed, feels like a scripting language, and has good support for interactive use, but can also optionally be separately compiled.

Reproducible

Reproducible environments make it possible to recreate the same Julia environment every time, across platforms, with pre-built binaries.

Composable

Julia uses multiple dispatch as a paradigm, making it easy to express many object-oriented and functional programming patterns. The talk on the Unreasonable Effectiveness of Multiple Dispatch explains why it works so well.

General

Julia provides asynchronous I/O, metaprogramming, debugging, logging, profiling, a package manager, and the ability to build binaries.

Open source

Julia is an open source project with over 1,000 contributors. It is made available under the MIT license. The source code is available on GitHub. Julia has a welcoming community accessible to all backgrounds.

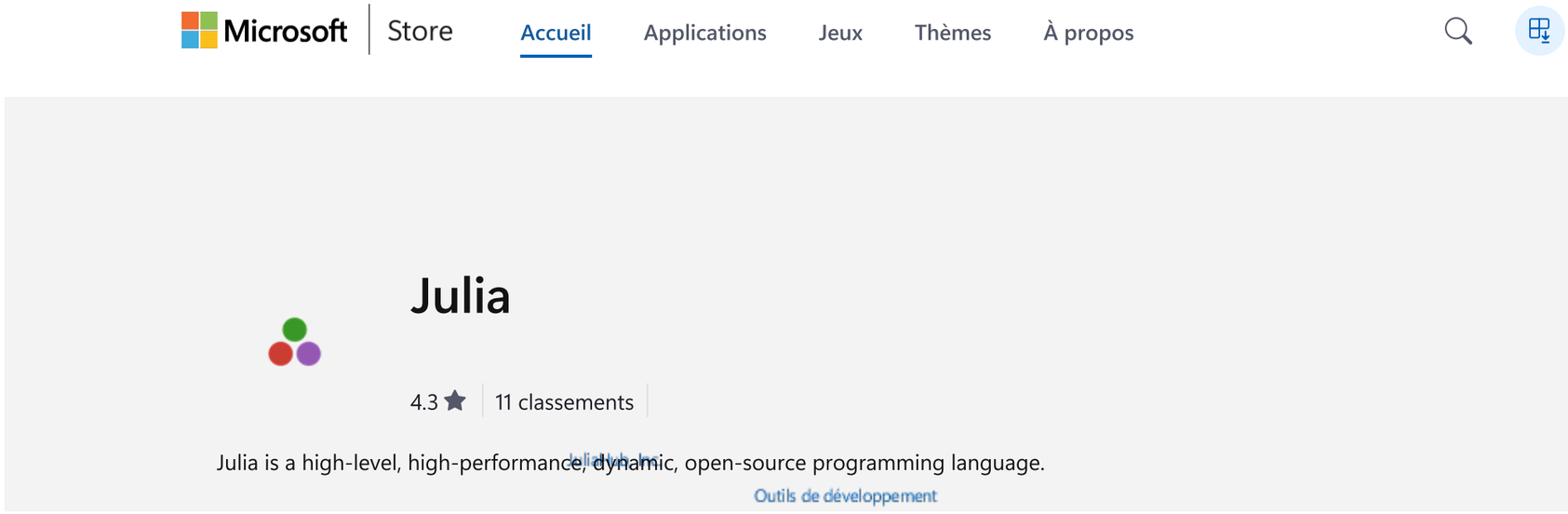
Multiple Dispatch

```
julia> struct Dog end; struct Cat end
```

Install

Windows

This page requires JavaScript.



Linux/Mac

Simply do

```
curl -fsSL https://install.julialang.org | sh
```

Both give you **`juliaup`**, the version manager:

```
juliaup add 1.13          # several versions side by side
juliaup default 1.13
juliaup update
julia +1.10 script.jl     # run with a specific one
```

Current stable: **v1.13.1**

The essentials

REPL (Terminal)

Good for

- Package `]`, environment management
- Shell mode `;`
- Quick look/computations
- Extensions: R console, TerminalPager.jl, etc.
- Can be connected to LLMs e.g. Kaimon.jl

VsCode (IDE)

Good for

- IDE features (IntelliSense, Highlighting, etc)
- Execute using REPL or script
- Development (with package, git, etc)
- Profiling
- Debugging (so so)
- Plot panel
- ...

Jupyter (notebook)

Good for

- Showing work
- Report like document
- Widely adopted
- JupyterLab

Not ideal for (it improved a lot)

- Development
- Sharing/Reproducibility e.g. `.ipynb` format!

Did you notice

Ju(lia) **Py**(thon) **R**

Pluto (notebook)

Good for

- Interactivity
- Sharing/Reproducibility e.g. `.jl` format!
- Teaching
- Presentation/Report like document

Bad for

- Heavy notebooks
- No IDE like features
- Development
- Pkg env sometimes

Reactive notebooks

14 / 10 / 2026

```
@bind date_start DatePicker(default=today())
```

224 ms

```
9.0
```

```
Ω^2
```

300 ns

```
date = range(; start=date_start, step=Day(1), length=length(t));
```

400 ns

Choose your color

Choose your frequency Ω = 3.0

```
t = 0.0:0.01:6.28
```

Other tools

Well established

- **Quarto** – one source → html, pdf, slides, website. Julia, Python and R in the same document.

Advanced Pluto usage

- **Pluto Desktop** → standalone app, no Julia install required (useful for students)
- **Space Station** → IDE like for Pluto
- **Advanced Pluto Notebook** extension brings Pluto into VSCode.

Newer, Julia only

- **KaimonSlate.jl**
- **Bonito Book**

Both worth a look

This talk

is a Pluto notebook rendered with PlutoSlides.jl. One file for the slides, the live demo and the hand-out.

Syntax

One-line function definitions and $\LaTeX$ symbols

```
φ₁ (generic function with 1 method)
```

```
φ₁(x) = exp(-x^2) / √(2π)
```

1.1 ms

```
😺 = 1
```

```
😺 = 1
```

20.7 µs

```
2
```

```
😺 + 😺
```

23.1 µs

```
λ (generic function with 1 method)
```

```
function λ(x, y)
    1+1
end
```

1.2 ms

```
cartesian (generic function with 2 methods)
```

```
cartesian(r, θ, ϕ=0) = r*sin(θ)*cos(ϕ), r*sin(θ)*sin(ϕ), r*cos(θ)
```

2.2 ms

Not just good looking: improve readability!

```
false
```

```
2 ∈ 4:7    # returns 'false'
```

25.1 µs

```
true
```

```
[1, 5] ⊆ 1:5  # returns 'true'
```

42.2 µs

```
f (generic function with 1 method)
```

```
f(x) = x[1]^2 + x[2]^2
```

1.1 ms

```
∇f (generic function with 1 method)
```

```
∇f(x) = [2x[1], 2x[2]]
```

1 ms

```
∇²f (generic function with 1 method)
```

Built-in arrays

```
A = 2×2 Matrix{Int64}:
 1  1
 1  0
```

```
A = [1 1;
     1 0]
```

45.8 μs

```
2×2 Matrix{Int64}:
 89  55
 55  34
```

```
A^10
```

31.2 μs

Broadcast

- Vectorization of operators and functions just with dot.
  - It is a language feature
  - It works with every function
- Vectorization is not **naturally** faster than loops but might be more convenient sometimes.

```
[2, 3, 4]
```

```
1 [1, 2, 3] .+ 1
```

159 ms

```
[2.71828, 7.38906, 20.0855]
```

```
1 exp.([1, 2, 3])
```

380 ms

```
[0.146763, 0.00730688, 4.92334e-5]
```

```
1 φ₁.([1, 2, 3])
```

110 ms

```
[1 ⇒ "a", 2 ⇒ "b", 3 ⇒ "c"]
```

```
1 [1, 2, 3] .=> ["a", "b", "c"]
```

233 ms

MetaProgramming (macros)

500 µs

Basically metaprogramming refers to a variety of ways a program has knowledge of itself or can manipulate itself. See docs for more details

In practice: cool macros

- Use to have simple call to other language
- `@time`, `@.`, `@ode_def` `@lie`, `@reaction_network`, ...

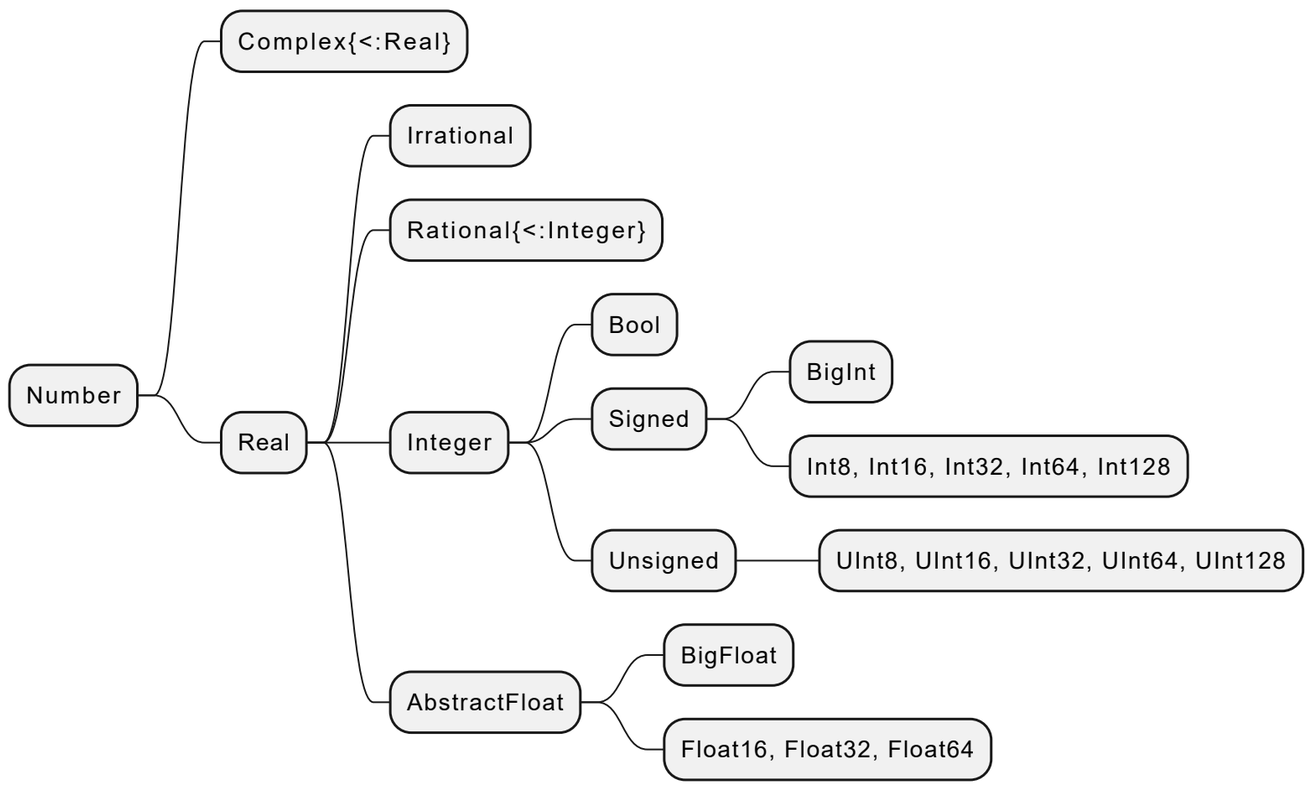
```
macro time(ex)
  return quote
    local t0 = time_ns()
    local val = $ex
    local t1 = time_ns()
    println("elapsed time: ", (t1-t0)/1e9, " seconds")
    val
  end
end
```

```
@btime rand(10^5);
```

```
  196.900 µs (3 allocations: 781.32 KiB)
```

12.5 sec

Types




```
abstract type Number end
abstract type Real          <: Number end
abstract type AbstractFloat <: Real end
abstract type Integer       <: Real end
abstract type Signed        <: Integer end
abstract type Unsigned      <: Integer end
```

[Ref]:

> See documentation on Types

Types are **not** mandatory but can be specified when needed!

true

```
Float16<:AbstractFloat
```

26.3 µs

3.0

```
(1+2.)::AbstractFloat
```

28.7 µs

3

```
(1+2)::Int
```

30.6 µs

500000500000

```
@btime sum(1:10^6)
```

Clever compiler

All the previous abstractions do not compromise efficiency, for two reasons.

Clever compiler (1/2)

The compiler infers variable types very well.

```
[2, 2, 2]
```

```
fill(2, 3)
```

35.3 µs

```
myfillmat (generic function with 1 method)
```

```
function myfillmat(x, n)
    v2 = fill(x, n)
    v = (v2)
    M = v * v'
    return M
end
```

1.8 ms

```
3×3 Matrix{Int64}:
 4  4  4
 4  4  4
 4  4  4
```

```
@time myfillmat(2, 3)
```

```
  0.000005 seconds (4 allocations: 224 bytes)
```

228 µs

```
@code_warntype myfillmat(2, 3)
```

```
MethodInstance for Main.var"workspace#32".myfillmat(::Int64, ::Int64)
  from myfillmat(x, n) @ Main.var"workspace#32" C:\Users\metivier\Dropbox\PC (2)\Documents\cours\2026_formation_Julia\notebook_1_Presentation.jl#==#aa3e7729-3b2e-4824-92af-09d15894b521:1
Arguments
```

Multiple Dispatch

For today

Multiple dispatch has **its own lecture** later this week. Here I only want you to see *why* it makes packages compose.

JuliaCon 2019 | The Unreasonable Effectiveness of Multiple Dispatch | Stefan Karpinski

What is multiple dispatch

Multiple dispatch (Polymorphism) in Julia

- Dispatch on **all** arguments of a function

Example: `f(x::Integer, y::AbstractFloat)` and `f(x::AbstractFloat, y::Integer)` can have different implementations.

- Dispatch depends on the **Type** of the argument not the class

1. **Generic algorithms** abstractly express high-level algorithms decoupled from implementation details
2. **Common Type** shared by many different packages

Example:

```
rand(Exponential(1.0))
rand(Exponential(1.0), 10)
rand(Normal(0, 1))
rand(Normal(0, 1), 1,5,6)
```

```
power(x, n::Integer) = prod(x for _ in 1:n)
power(x, n::AbstractFloat) = exp(n * log(x))
```

Type/Class sharing

Class Based Object Oriented `x.f(y)`

Methods **live inside** the `class` definition. To add a new method on a type you do not own, you must either

1. **Edit the original class** (you need write access & must maintain it), or
2. **Inherit from it** with a new class (your method won't apply to the original type nor its other subclasses)

```
# Dog comes from some library you imported
class Dog:
    def speak(self):      # method trapped inside the class
        return "Woof"

d = Dog()
d.speak()                 # ok
d.fetch()                 # AttributeError: you cannot add `fetch` from outside
```

'Solutions'

1. Convince the authors to add your method (they'll have to maintain it & cannot easily remove it)
2. Subclass into `MyDog` & your method won't apply to other `Dog`s
3. Give up on dispatch: use an external function `fetch(d)` instead of `d.fetch()`
4. Give up code sharing

In Julia functional programming `f(x,y)`

Functions live **outside** types: you can add methods on a type **after** it is defined, even one you do not own. That's it!

Expressiveness



[from]:

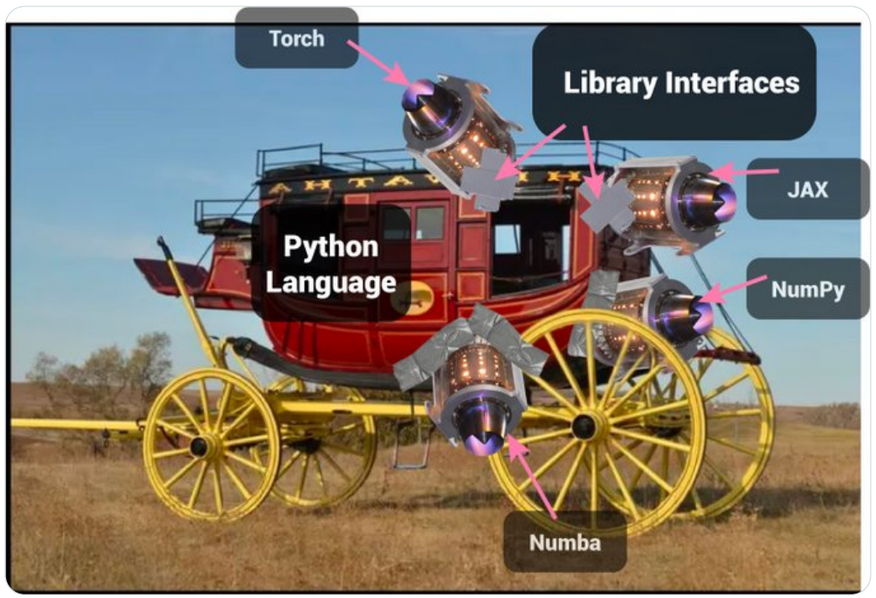
> Tim Holy this video, that notebook and this paper

Composability

Miles Cranmer
@MilesCranmer · **Follow**

The more I use Julia, the more Python and its numeric libraries look like a Victorian-era stagecoach with jet engines duct-taped to it, each pointing a different direction (=mutually incompatible).

It's such a weird ecosystem, and makes it so much harder for users to contribute.



6:50 PM · Nov 7, 2022

572 Reply Copy link to post

Read 29 replies

In Julia

One language + type sharing + multiple dispatch

⇒ Amazing, unexpected combination and code reuse

⇒ Improving a package can benefit the **whole** ecosystem

What is NOT code reuse?

Recoding basic stuff because of incompatible frameworks

`tensorflow.random.uniform`, `numpy.random.rand`, `torch.rand`, `jax.random.uniform`

Multiple dispatch showcase

Custom function `mynorm` that computes the sum of squares of a vector.

```
mynorm (generic function with 1 method)
```

```
mynorm(v) = sum(abs2, v)
```

908 µs

```
v = [1, 2, 3]
```

```
v = [1, 2, 3]
```

156 µs

```
14
```

```
mynorm(v)
```

27.6 µs

Logarithmic numbers to prevent numerical underflow (LogarithmicNumbers.jl):

```
v_log = [+exp(0.0), +exp(0.693147), +exp(1.09861)]
```

```
v_log = LogFloat64.(v)
```

91.8 ms

```
+exp(2.639057329615259)
```

```
mynorm(v_log)
```

60.7 µs

Numbers with physical units (Unitful.jl):

```
v_unit = Unitful.Quantity{Float64, L, Unitful.FreeUnits{(m,), L, nothing}}[
             1: 0.001 m
             2: 2.0 m
             3: 3.0 m
         ]
```

```
v_unit = [Float64(1)u"mm", 2u"m", 3u"m"]
```

377 µs

```
13.000001000000001 m^2
```

```
mynorm(v_unit)
```

700 ns

Numbers with uncertainties (Measurements.jl)

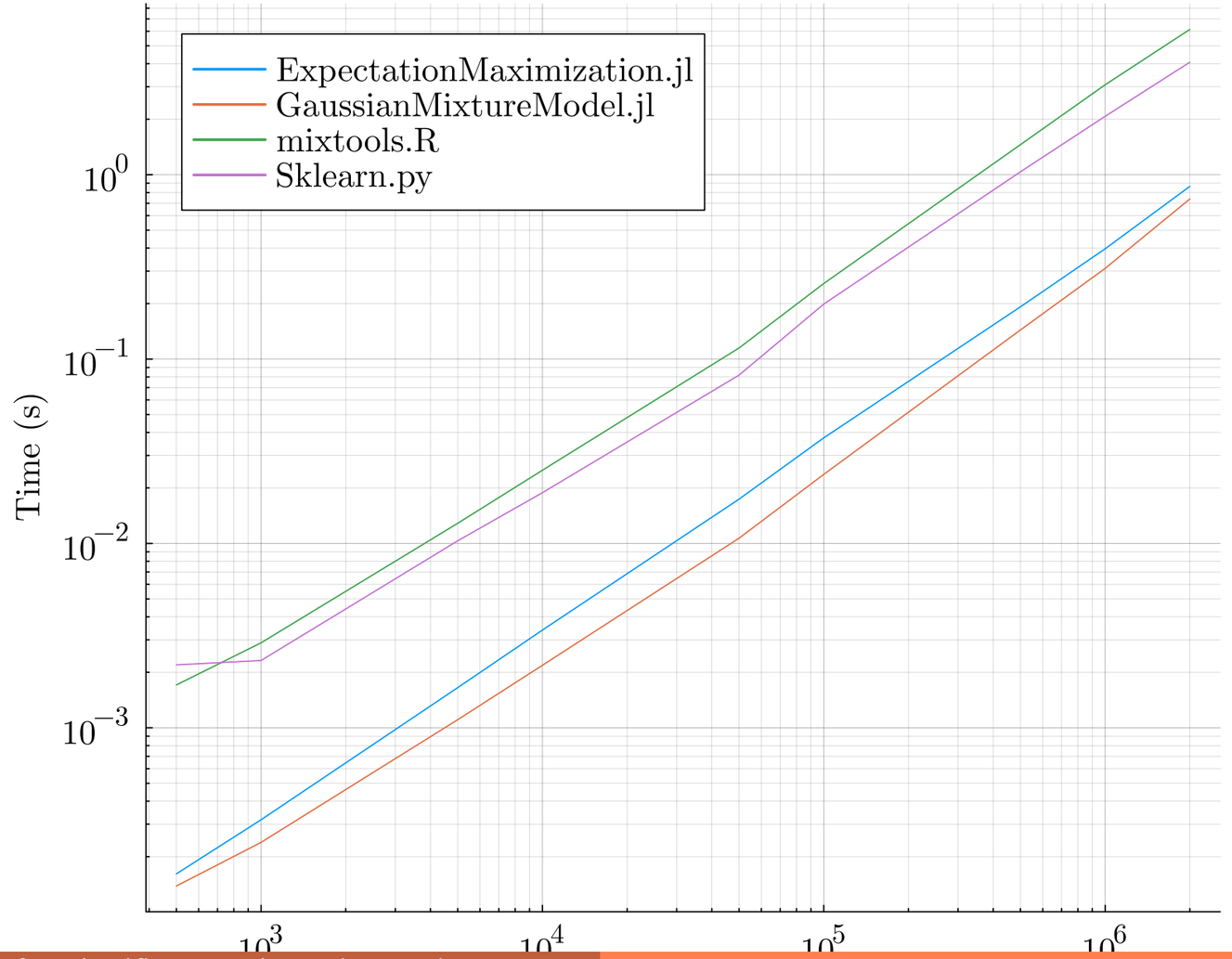
Generic & Efficient code

Julia is near **Pareto-optimal** in terms of **lines of code** vs **efficiency**.

- Computer Language Benchmarks Game
- Energy efficiency of programming languages
- ChatGPT works best in Julia for LLM Code Generation over Python/R (see Figure 1).

Personal experience: The Expectation Maximization algorithm

ExpectationMaximization.jl versatile + fast



Ecosystems

An ecosystem of small things

Two populations

- A few **big reference ecosystems/organizations**: SciML, JuliaStats, JuliaData, JuliaGraphs, JuliaGPU, Makie, JuMP...
- Hundreds of **tiny packages** doing exactly one thing, e.g. ToeplitzMatrices.jl

The philosophy

One package = **one mathematical object**, or one algorithm.

Small ⇒ easy to test, to document, to review, to trust. And it composes with the rest precisely *because* it only talks about its own object.

The consequence

Your small package can be useful to someone else.

Statistics

Statistics & inference

- `StatsBase.jl`, `Distributions.jl`
- built on top: `Copulas.jl`, `ExpectationMaximization.jl`, `AdditionalDistributions.jl`, ...
- `Turing.jl` – Bayesian inference
- `HiddenMarkovModels.jl`, `GLM.jl`
- `MLJ.jl` – one interface over many ML packages

```
dist = Distributions.Exponential{Float64}(θ=5.0)
```

```
dist = Exponential(5)
```

55.9 µs

```
5.0
```

```
mean(dist)
```

100 ns

```
data =
```

```
[9.30039, 2.40121, 8.35925, 10.9218, 4.47314, 10.6612, 8.53065, 0.0821861, 5.6222, 0.470839, 3.40354, 17.7185, 2.21323, 2.14236, 1.02458, 3.11874, 17.0
```

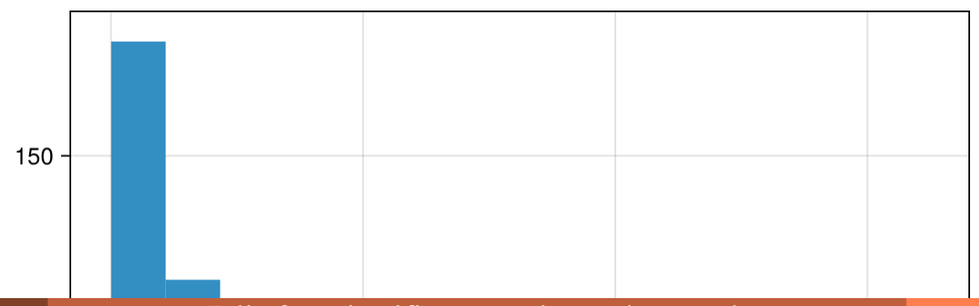
```
data = rand(dist, 500)
```

529 µs

```
0.07069093639175604
```

```
pdf(dist, 5.2)
```

1.7 µs



Automatic Differentiation

Definition

- Automatic differentiation is a technique to compute the derivative of a function by applying the chain rule to the elementary operations that compose it.
- It is not symbolic differentiation, nor numerical differentiation.

⇒ In Julia most of the code can be automatically differentiated out of the box.

Extremely useful for optimization, machine learning, etc.

Example: Babylonian square root algorithm

```
mysqrt (generic function with 1 method)
```

```
function mysqrt(x)
    y = x
    for k in 1:5
        y = 1/2 * (y + x / y)
    end
    return y
end
```

4.5 ms

```
1.414213562373095
```

```
mysqrt(2)
```

100 ns

What if I want to compute its derivative?

Symbolic differentiation

▶ [x]

Which AD backend?

You just wrote `Dual` by hand. In practice you pick one:

package	mode	good for
`ForwardDiff.jl`	`forward`	`few inputs – exactly the Dual above`
`Zygote.jl`	`reverse`	`many inputs, source-to-source`
`Enzyme.jl`	`both`	`LLVM level, very fast, handles mutation`
`Mooncake.jl`	`both`	`pure Julia, newer`

```
value_and_gradient(f, AutoForwardDiff(), x)
value_and_gradient(f, AutoEnzyme(), x)
```

One interface: DifferentiationInterface.jl

`DifferentiationInterface.jl` wraps all of them. Swap `AutoForwardDiff()` for `AutoEnzyme()` and re-run – your code does not move.

Downside

This much choice is fairly **unique to Julia**, and it is also the hard part: supporting several of them in one code base costs.



SciML: Differentiable Modeling and Simulation Combined with Machine Learning

The SciML organization is a collection of tools for solving equations and modeling systems developed in the Julia programming language with bindings to other languages such as R and Python. The organization provides well-maintained tools which compose together as a coherent ecosystem. It has a coherent development principle, unified APIs over large collections of equation solvers, pervasive differentiability and sensitivity analysis, and features many of the highest performance and parallel implementations one can find.

Scientific Machine Learning (SciML) = Scientific Computing + Machine Learning

Where to Start?

- Want to get started running some code? Check out the Getting Started tutorials.
- What is SciML? Check out our Overview.
- Want to see some cool end-to-end examples? Check out the SciML Showcase.
- Curious about our performance claims? Check out the SciML Open Benchmarks.
- Want to learn more about how SciML does scientific machine learning? Check out the SciML Book (from MIT's 18.337 graduate course).
- Want to chat with someone? Check out our chat room and forums.
- Want to see our code? Check out the SciML GitHub organization.

And for diving into the details, use the bar on the top to navigate to the submodule of interest!

package	what it does
`DifferentialEquations.jl`	`ODE/DAE/SDE/etc. solvers`
`ModelingToolkit.jl`	`symbolic model → simplification`
`Optimization.jl`	`one front-end over ~20 optimizer backends`

Outside SciML

Optimization

- `JuMP.jl` – algebraic modeling for LP/MILP/NLP/conic, its own DSL and its own solver list (HiGHS, Ipopt, Gurobi, ...)
- `Optim.jl` – the classic unconstrained/box-constrained solvers, pure Julia
- `Manopt.jl` + `Manifolds.jl` – optimization on manifolds
- `NLopt.jl`, `Ipopt.jl` – thin wrappers over the reference C/Fortran libraries

PDE

- `Ferrite.jl` – finite elements, low level, you assemble yourself
- `Gridap.jl` – FEM written from the weak form, close to the maths
- etc.

Composability in action: Bayesian inference of ODE parameters

The task

Given noisy data (t_i, y_i), **recover the parameters** p of a dynamical system.

Example – predator/prey (Lotka–Volterra), $p = (\alpha, \beta, \gamma, \delta)$:

$$\frac{\mathrm{d}u}{\mathrm{d}t} = f(u, p, t)$$

Fit the solution to the data, *and* quantify uncertainty on p → **Bayesian inference**.

The monolithic nightmare

One package would need to be expert in **all** of:

- ODE solvers
- optimization (not just derivative-free methods e.g. Nelder-Mead)
- automatic differentiation
- Bayesian sampling (MCMC)

(+ likely some C/C++ glue) → hard to maintain

The Julia way: assemble specialists

Each package does **one** thing well, and they **compose** out of the box:

Package	Specialty
`OrdinaryDiffEq.jl`	`solve the ODE`
`Turing.jl`	`Bayesian inference (MCMC)`
`Optimization.jl`	`find the best fit`
`ForwardDiff / Zygote`	`differentiate the solver`

Why it just works

Shared types + multiple dispatch: Turing knows how to differentiate through the ODE solver, the solver accepts AD

⇒ **No glue code, no rewrite.** Swap optimization for Bayesian by changing a few lines.

Calling other languages

Interfaces with C(++), Python and R are solid and easy to use.

Coding Stochastic Weather Generators: Challenges and Perspectives

AUTHOR
David Métivier

PUBLISHED
December 8, 2025

The notebook is the companion to the talk Coding Stochastic Weather Generators: Challenges and Perspectives given at the SWGEN 2025 - Conference on Stochastic Weather Generators.



SWGEN 2025 - Conference on Stochastic Weather Generators
2-4 Dec 2025 Grenoble (France)

Login

It showcases two challenges encountered when developing Stochastic Weather Generators (SWGs):

- Simulating SWGs efficiently, through a benchmark of HMM simulation in Julia, R, (Python) and C. It uses LLMs to generate the code in each language to mimic what a typical student/scientist might write.
- Fitting SWGs. It uses a Matérn Gaussian Random Field (GRF) as a running example to illustrate parameter estimation via gradient-based optimization with automatic differentiation, finite difference and gradient free methods e.g. Nelder-Mead.

```
cd(@__DIR__)
import Pkg
Pkg.activate(".");
using PrettyTables
```

```
  Activating project at `C:\Users\metivier\Dropbox\PC (2)\Documents\GitHub\Pluto_export\SWG_coding_challe
```

```
# import PythonCall #if you want to test Python version
import RCall
```

Data

Tables.jl: the ecosystem in one interface

A tiny package whose only job is to define **what a table is**. Everything below speaks it, so they are interchangeable:

`DataFrames.jl`, `CSV.jl`, `Arrow.jl`, `DuckDB.jl`, `XLSX.jl`, etc.

Reference ecosystem: JuliaData.

In practice

```
df = CSV.read("runs.csv", DataFrame)
Arrow.write("runs.arrow", df)
DuckDB.query(con, "SELECT ...") |> DataFrame
```

Any producer, any consumer, **no conversion code** – and no copy at all for the columnar ones.

Same pattern as `Optimization.jl` and `DifferentiationInterface.jl`.

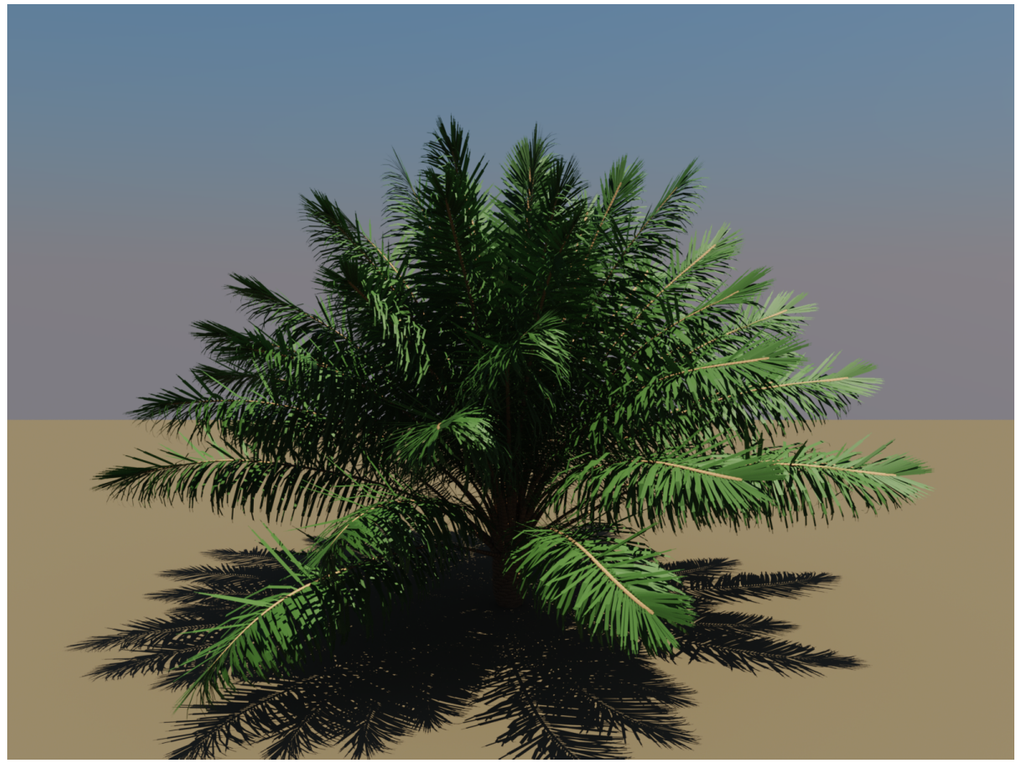
	date	mesh	dt	err
1	2022-02-03	16	0.01	0.041
2	2022-02-04	32	0.005	0.0103
3	2022-02-06	64	0.0025	0.0026
4	2022-02-08	128	0.00125	0.00064

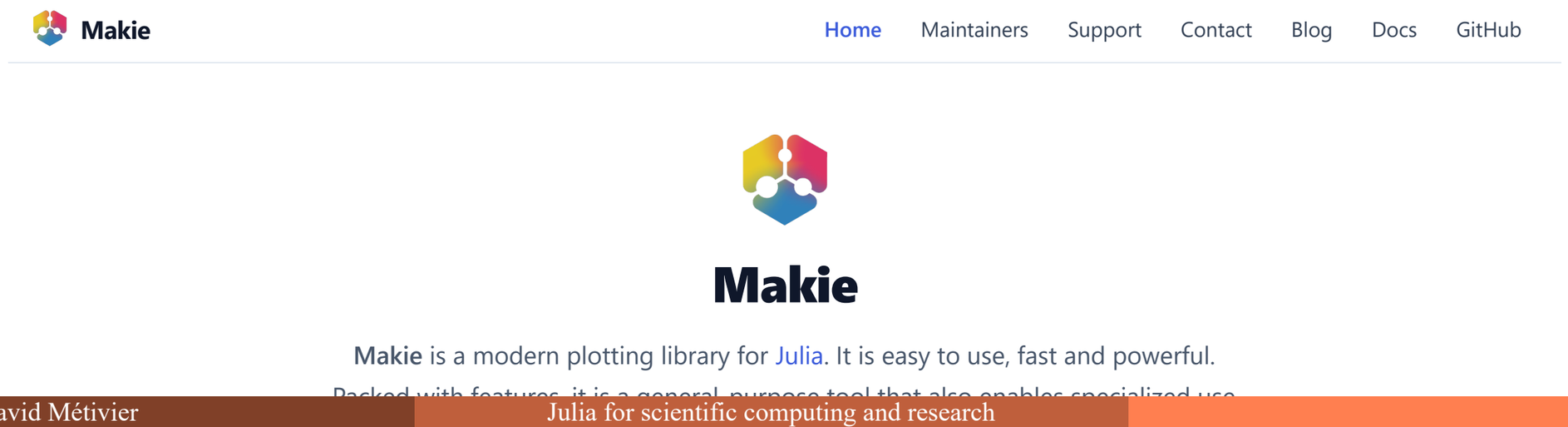
```
DataFrame(
    date=[Date(2022, 2, 3), Date(2022, 2, 4), Date(2022, 2, 6), Date(2022, 2, 8)],
    mesh=[16, 32, 64, 128],
    dt=[1e-2, 5e-3, 2.5e-3, 1.25e-3],
    err=[4.1e-2, 1.03e-2, 2.6e-3, 6.4e-4],
)
```

Plotting

Example **Makie.jl**: One API and lot of backends (static, interactive, web, raytracing, GPU, maps, ...)

> “[...] The fact that a single plotting ecosystem can cover the entire spectrum from high-performance 3D scenes to publication-ready figures is just exceptional. [...]”
>
> **Rémi Vezy, CIRAD, developer of PlantGeom.jl for plant-scale digital twins**



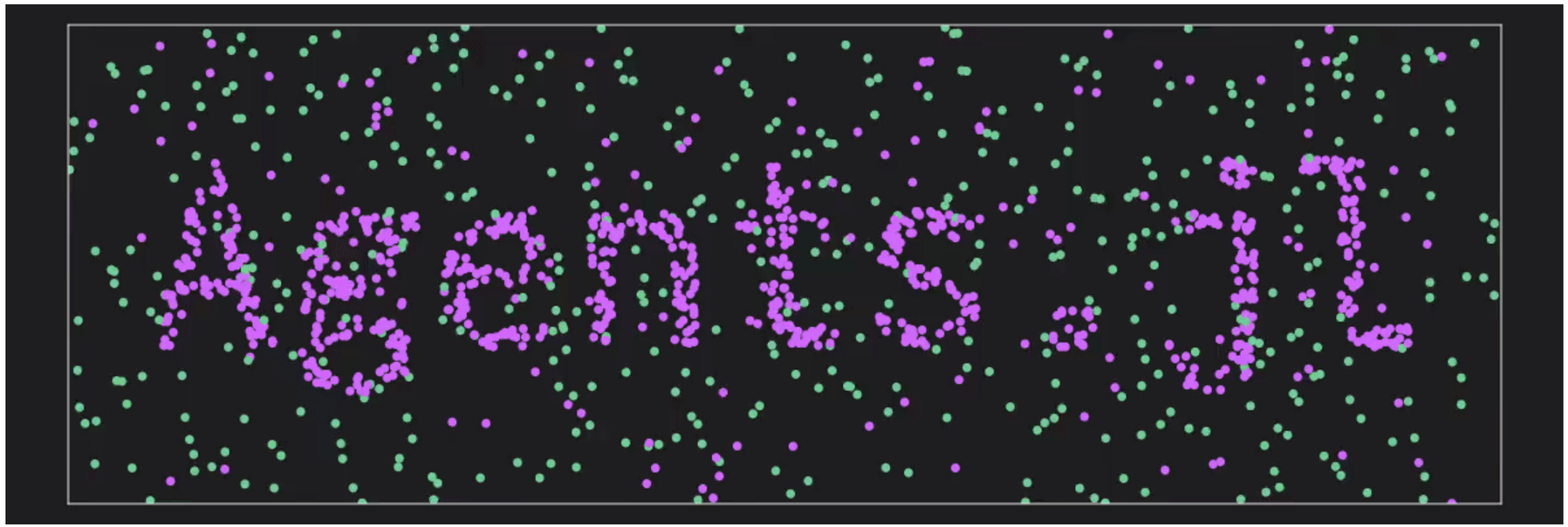


Agents

Introduction

GitHub

Agents — Module



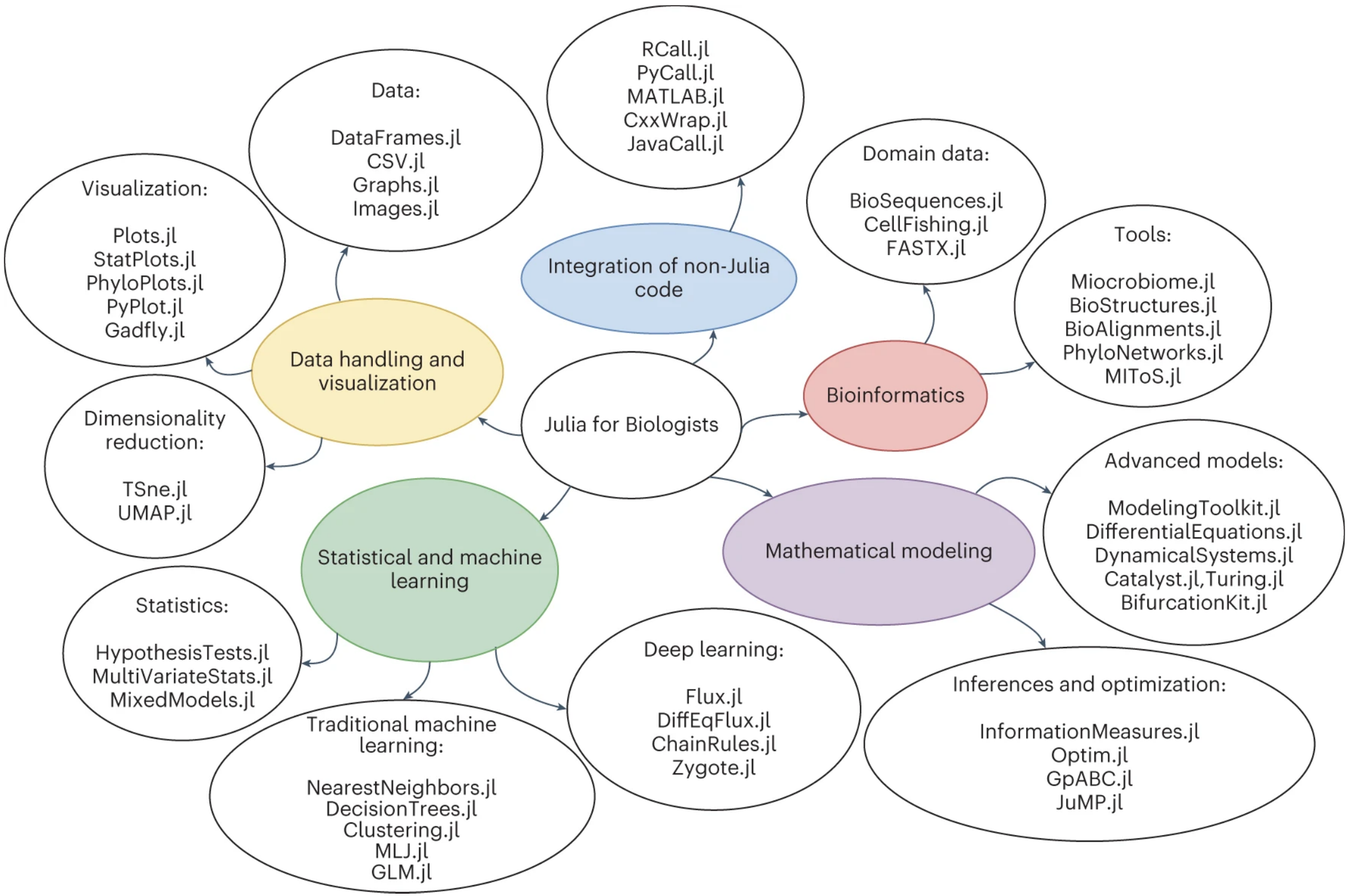
docs stable | DOI 10.1177/00375497211068820 | CI failing | codecov 85% | tested with Aqua.jl | Downloads 28,039

Agents.jl is a pure Julia general-purpose framework for agent-based modeling (ABM): a computational simulation methodology where autonomous agents react to their environment (including other agents) given a predefined set of rules. Some major highlights of Agents.jl that set it apart from other similar frameworks are:

1. It is simple: has a very short learning curve and requires writing minimal code
2. Has an extensive interface of thousands of out-of-the box possible agent actions
3. It is fast (typically faster than established competitors)

For Biologists

This paper sells Julia pretty well for Biologists (and others)



Web application

Community

Julia features favor a lively community

Note

1. Composability ⇒ **Code reuse**, connected community
2. **One unique language favors contributions** (no need to both know C/C++ and Python/R)
3. One unique language helps to **read other codes**. Example `Exponential` distribution from `Distributions.jl`

- You can actually help improve both the **ecosystem** (packages) and the **language** itself
- Open source dev will teach you good programming practices
- Easier with LLMs
- The community is very helpful

Fun Fact

I registered the 9,999th package `PeriodicHiddenMarkovModels.jl`. A few minutes later came the 10,000th one

Events

Every year: JuliaCon

- A great conference with plenty of topics
- Introductory tutorials
- Remote option

European JuliaCon

- JuliaCon Local Eindhoven 2023
- JuliaCon Local Paris 2025

France

By Pierre Navaro

- Julia and Optimization Days (since 2022)
- CNRS mailing list newsletter
- Website
- Monthly talk Café Julia

Julia4HPC : Formation au Langage Julia pour le Calcul Haute Performance 12-16 oct. 2026 Frejus

Packages

What is inside a package

```
MyPackage.jl/
├── Project.toml            # name, uuid, version, deps, [compat]
├── src/
│   └── MyPackage.jl        # the module: using/import, include(...), export
├── test/
│   └── runtests.jl         # ] test
├── docs/                   # Documenter.jl -> a website
├── ext/                    # extensions: loaded only if another package is
└── .github/workflows/      # CI, doc build, TagBot
```

The folder has `.jl`, the module does not

`MyPackage.jl/` is the **repository**, `module MyPackage` is the **code**.

Templates

You do not write those files by hand.

`PkgTemplates.jl`	`the classic, plugins à la carte`
`BestieTemplate.jl`	`opinionated (:tiny, :light, full). Applies to an existing package too, one feature at a time`

- Official guide: developing a package
- The BestieTemplate FAQ explains what **every** generated file is for – read it once, it saves hours

Exercise: Fitting neural ODE with four handmade & independent packages

Groups of four -- one package each

who	package	provides
1	`FrejusJuliaAD0`	`derivatives, by dual numbers`
2	`FrejusJuliaODE0`	`explicit Euler / RK4`
3	`FrejusJuliaNeuralNetworks0`	`dense layers, chain, flat parameter vector`
4	`FrejusJuliaOptimization0`	`Newton, Gauss-Newton, gradient descent`

At the end the four are assembled, **unchanged**, to train

$$\frac{\mathrm{d}u}{\mathrm{d}t} = \mathrm{NeuralNetwork}(u)$$

on data.

The two rules

1. No package depends on any other

This is what lets the four of you work **in parallel**.

2. Write generic code

No `::Float64` in a signature, no `zeros(3)` where the element type should come from the input. Ask for the least you need.

Rule 2 is what makes rule 1 possible

The AD package will push its `Dual` numbers through the ODE solver and through the network – code that has **never heard of a dual number**.

Written generically, it works.

That is the lesson of the afternoon. See the `README.md` of the exercise.

Conclusion

Conclusion

One language

- **Multiple dispatch** + a single shared type system → naturally reusable, composable code
- Reading, contributing to, and extending other people's packages requires *no* language switch

⇒ Favors **collaborative work** and a healthier open-source ecosystem

Efficiency without sacrificing expressiveness

- JIT compilation → C-level speed with high-level syntax
- **Automatic differentiation out of the box** — not just for neural networks, but for *any* Julia code (ODEs, physical models, custom algorithms…)
- Personal example: training a neural ODE for lake temperature forecasting — one coherent Julia pipeline, no glue code from data loading to model training to visualization

A rich, extensible ecosystem

- State-of-the-art packages: DataFrames.jl, DifferentialEquations.jl, Symbolics.jl, JuMP.jl, Makie.jl…
- New packages compose easily with existing ones (no rewrite/glue code needed)
- Seamless interoperability with **C, Python, R**, … — keep your irreplaceable libraries

A great community

- One shared language lowers the barrier to contribution
- Active forums, JuliaCon, local events (Café Julia, France…)

Resource

- Modern Julia Workflows: A must-read for beginner and advanced users.
- Julia Academy and here for the first lecture video
- DataFrames.jl Cheat Sheet, also look at DataFramesMeta.jl
- Julia Data Science: Big overview aimed at data science
- doggo dot jl Youtube videos and associated repo: Tutorials over plenty of basic and advanced topics.

I should definitely add more!

Appendix

Packages used

```
begin
    using BenchmarkTools
    # using Graphs
    using IntervalArithmetic: interval
    using DataFrames
    using Dates

    using LinearAlgebra
    using LogarithmicNumbers
    using Measurements
    using Pkg
    using Distributions
    using Symbolics # Symbolic
    using Unitful # Physics unit
end
```

13.4 sec